

Chapter - 11.2.

Application of trees.

① Binary Search Tree: (BST)

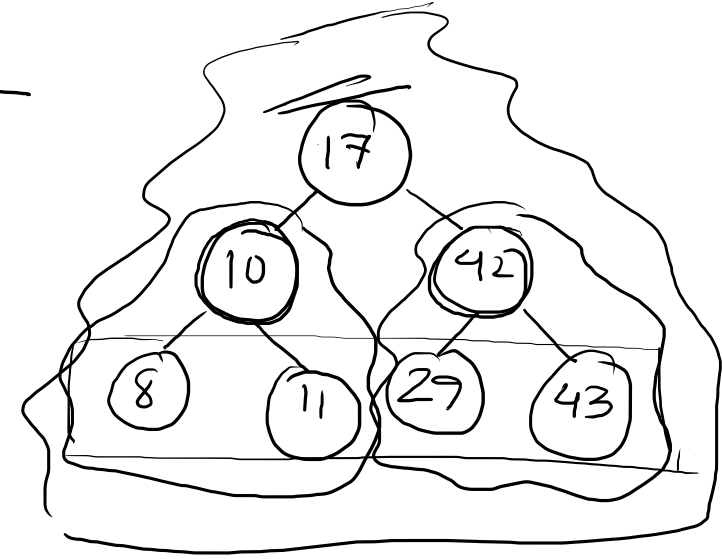
- $\text{value}_{\text{child}} < \text{parent} \rightarrow \text{left node}$
- $\text{value}_{\text{child}} > \text{parent} \rightarrow \text{right node}$

Construct BST with 17, 10, 42, 11, 8, 29, 43

② Decision tree $\rightarrow$ m-ary tree.

$\rightarrow$ rooted tree

$\rightarrow$ internal vertices represent a decision



④ Prefix codes.

- Encode 26 letters of EA using bit string
 - $2^5 = 32 > 26 \rightarrow |\text{bitstring}| = 5$
 - Can we encode efficiently
- less memory?
less Tx time?

⑤ Consider encoding using varying length bit-strings.

Start and end

represent frequent letters with shorter bit strings.

$\begin{matrix} e \rightarrow 0 \\ a \rightarrow 1 \\ t \rightarrow 01 \end{matrix} \rightarrow \boxed{0101} \rightarrow \begin{cases} tt \\ eat \\ tea \\ eaea \end{cases} \rightarrow \text{ambiguity}$

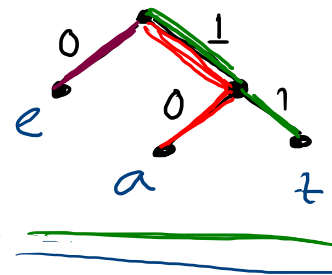
- to remove ambiguity $\rightarrow$ bit string for a letter never occurs as the first part of another letter. $\rightarrow$ Prefix codes

$\begin{matrix} e \rightarrow 0 \\ a \rightarrow 10 \\ t \rightarrow 11 \end{matrix} \rightarrow \boxed{0101} \rightarrow eat$

- Encode using rooted binary tree

- 0 $\rightarrow$ left edge
- 1 $\rightarrow$ right edge

- chars $\rightarrow$ leaf nodes.



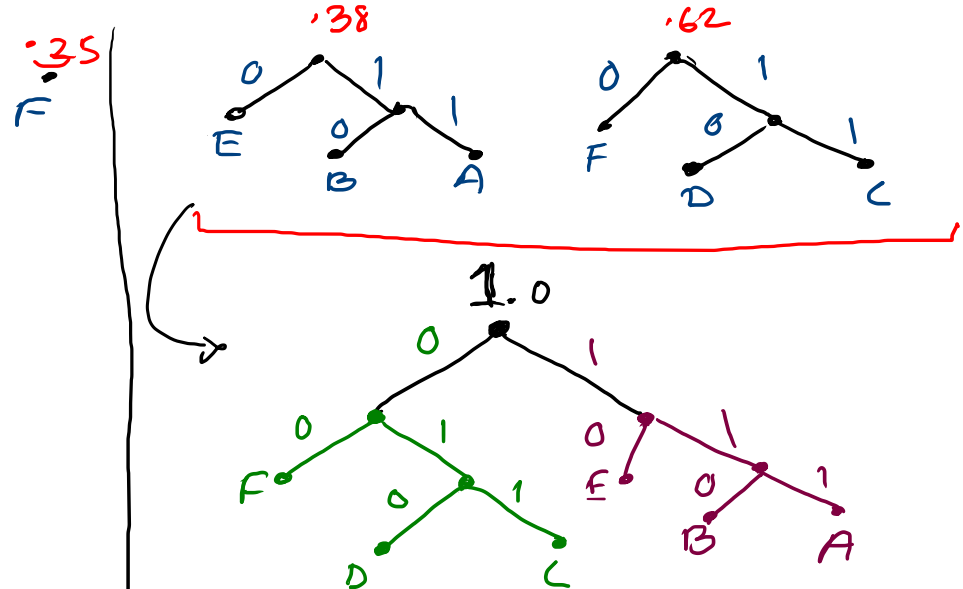
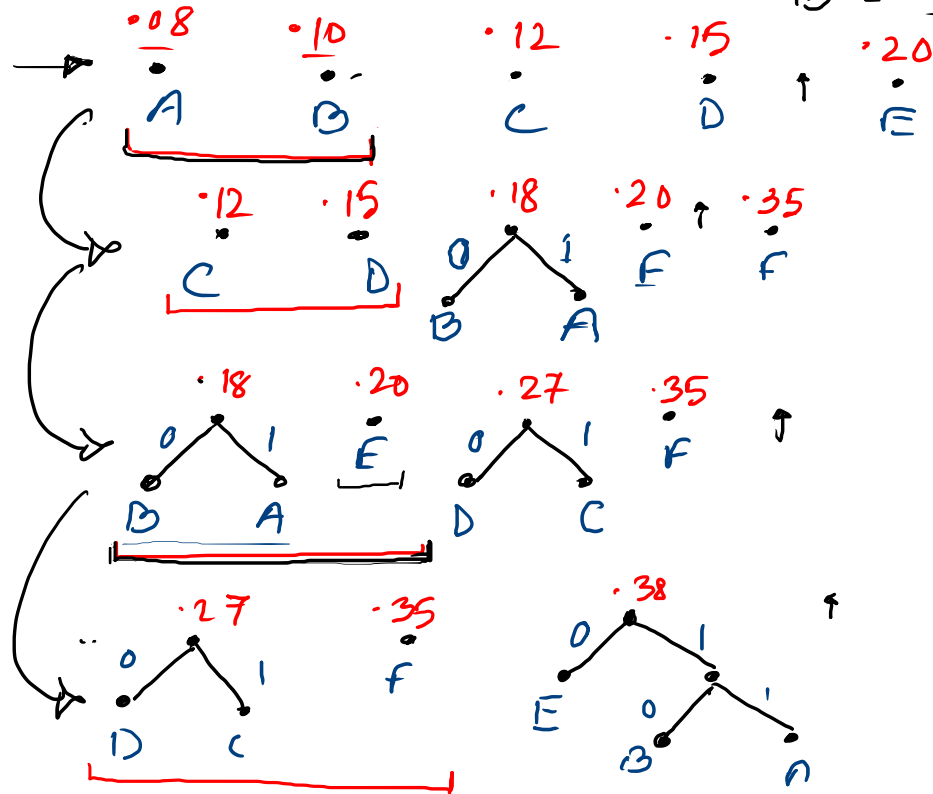
⑨ Huffman coding

- INPUT: Frequency/Probabilities of symbols in a string
- OUTPUT: A prefix code with the fewest possible bits
- GOAL: Given the frequencies, generate the rooted binary tree.

- Generate HC for $\rightarrow A = .08$
 $B = .10$

$C = .12$
 $D = .15$

$E = .20$
 $F = .25$



$A = \underline{111}$
 $B = \underline{110}$

$C = \underline{011}$
 $D = \underline{010}$

$E = \underline{10}$
 $F = \underline{00}$

* Game trees.

- Vertices → current state of game
- Edges → legal moves.
- Leaves → final outcome of the game.
- root → starting point.
- { vertex in even level → 1st player
" " " odd " → 2nd player



Game of Nim



Tic-Tac-Toe